

GraphQL and API federation

Assignment #7

Software Systems Architecture

1MEIC02 T3

Gonçalo Gonçalves da Costa Pinto
José Diogo Alves Granja
Manuel Mo
Pedro Rafael Correia Borges
Ricardo Miguel Yang

1. What is GraphQL

GraphQL is a query language for APIs and a runtime for fulfilling those queries with your existing data. Unlike REST, which is resource-oriented (where the server defines the response), GraphQL is **client-driven**.

- **The Single Endpoint:** Clients send a single POST request to one `/graphql` endpoint.
- **The Type System:** It uses a Schema Definition Language (SDL) to define exactly what data is available.
- **Precision Fetching:** It eliminates **over-fetching** (getting more data than you need) and **under-fetching** (making multiple calls to get related data).

Core Concept: Think of REST as a vending machine (fixed items) and GraphQL as a buffet (you pick exactly what goes on your plate).

2. Pattern 1: The GraphQL Monolith

This is the standard starting point for most companies. All schemas, types, and data-fetching logic (resolvers) live in a single server.

- **Best For:** Startups and small-to-medium teams.
- **Advantages:**
 - * **Simplicity:** One codebase to manage and one server to deploy.
 - **Performance:** No internal network latency; data is fetched within a single process.
 - **Unified Graph:** All data relationships are easily defined in one place.
- **The "Wall":** As the organization grows (e.g., 50+ developers), the Monolith becomes a **bottleneck**. Teams face constant merge conflicts in one giant schema file, and deployment slows down because everyone is tied to the same release cycle.

3. Pattern 2: API Federation (The Evolution)

API Federation is an architectural pattern that allows you to split your GraphQL API across multiple teams while still presenting a **single unified interface** to the client.

- **The Supergraph:** A "Gateway" (or Router) sits in front of multiple **Subgraphs** (Microservices).
- **Distributed Ownership:** The "Product Team" owns the Product Subgraph; the "Users Team" owns the User Subgraph. They develop and deploy independently.

- **Declarative Composition:** The Gateway automatically "stitches" these Subgraphs together into one seamless schema. The client has no idea the data is coming from different services.
- **The Benefit:** It combines the **autonomy of microservices** with the **usability of a monolith**.

4. When to use it

Why use GraphQL at all?

- **Complex Data:** Your UI needs deeply nested data (e.g., *User > Orders > Products > Reviews*) in one request.
- **Mobile-First:** You need to save bandwidth by fetching only the necessary fields, eliminating over-fetching.
- **Frontend Speed:** Frontend teams can iterate on UI changes without waiting for backend API changes.

When NOT to use it (The Trade-offs):

- **Simple CRUD APIs:** If your application mostly handles basic data retrieval without complex relationships, setting up GraphQL is over-engineering. Stick to REST.
- **Heavy Caching Needs:** REST handles HTTP-level caching natively. Because GraphQL usually runs through a single POST endpoint, standard network-level caching is significantly harder to implement.

Choosing your Pattern

If you have...	Use this Pattern
A single team / Small App	Monolith: Keep it simple. Don't add infrastructure complexity (Gateways/Routers) until you actually feel the pain of a growing team.
Many teams / Microservices	Federation: Use this when you need to scale. It allows teams to move fast and own their data without breaking the "Single Entry Point" for the frontend.

5. Conclusion

As GraphQL has matured, it has transitioned from an experimental technology to an industry standard for data-driven organizations. Its adoption patterns typically mirror a company's organizational growth.

- **The Pioneers (Schema Complexity):**
 - **Meta (Facebook/Instagram):** Created GraphQL to handle the massive, interconnected social graph of billions of users.
 - **GitHub:** Provided one of the first major public GraphQL APIs to offer third-party developers more precision than REST allowed.
- **The Federation Leaders (Microservices at Scale):**
 - **Netflix:** Uses GraphQL Federation to stitch together hundreds of microservices into a single "Supergraph," powering their UI across thousands of device types.
 - **Airbnb:** Leverages a federated architecture to allow independent teams to contribute to a unified data layer without deployment bottlenecks.
- **The Scale-ups (Performance & Data Access):**
 - **Shopify:** Uses GraphQL to power high-performance, customizable e-commerce storefronts.
 - **Amazon (AWS & Selling Partner API):** Provides GraphQL as a managed service (AppSync) and uses it to simplify complex retail data for third-party sellers.

Ultimately, GraphQL and API Federation solve a critical tension in software engineering. They allow backend teams to stay decoupled and move fast using microservices, while giving frontend teams the simple, unified "buffet" they need to build great user interfaces. It is the bridge that lets companies scale their organization without breaking their product.